

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
/* Treap Node*/
```

```
typedef struct Node {
```

```
    int task_id;
```

```
    int priority;
```

```
    struct Node *left;
```

```
    struct Node *right;
```

```
} Node;
```

```
/* Create a new node*/
```

```
Node* createNode(int task_id, int priority) {
```

```
    Node* newNode = (Node*)malloc(sizeof(Node));
```

```
    newNode->task_id = task_id;
```

```
    newNode->priority = priority;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
/* Right Rotation */
```

```
Node* rotateRight(Node* root) {  
    Node* newRoot = root->left;  
  
    root->left = newRoot->right;  
    newRoot->right = root;  
  
    return newRoot;  
}
```

/* Left Rotation*/

```
Node* rotateLeft(Node* root) {  
    Node* newRoot = root->right;  
  
    root->right = newRoot->left;  
    newRoot->left = root;  
  
    return newRoot;  
}
```

/* Insert Task */

```
Node* insert(Node* root, int task_id, int priority) {  
    /* Empty tree */  
    if (root == NULL)  
        return createNode(task_id, priority);
```

```
/* BST insertion */
```

```
if (task_id < root->task_id) {
```

```
    root->left = insert(root->left, task_id, priority);
```

```
    /*
```

```
        Max Heap property violated.
```

```
        Child has higher priority than parent.
```

```
    */
```

```
    if (root->left->priority > root->priority)
```

```
        root = rotateRight(root);
```

```
}
```

```
else if (task_id > root->task_id) {
```

```
    root->right = insert(root->right, task_id, priority);
```

```
    /*
```

```
        Max Heap property violated.
```

```
    */
```

```
    if (root->right->priority > root->priority)
```

```
        root = rotateLeft(root);
```

```
} else {
```

```
    printf("Task ID %d already exists!\n", task_id);
```

```
}
```

```
return root;
```

```
}
```

```
/* Delete Task */
```

```
Node* deleteNode(Node* root, int task_id) {
```

```
    if (root == NULL)
```

```
        return NULL;
```

```
/* Search using BST property */
```

```
if (task_id < root->task_id) {
```

```
    root->left = deleteNode(root->left, task_id);
```

```
}
```

```
else if (task_id > root->task_id) {
```

```
    root->right = deleteNode(root->right, task_id);
```

```
}
```

```
else {
```

```
    /* No left child */
```

```
    if (root->left == NULL) {
```

```
        Node* temp = root->right;
```

```
        free(root);
```

```
        return temp;
```

```
    }
```

```
/* No right child */
```

```
if (root->right == NULL) {
```

```

    Node* temp = root->left;

    free(root);

    return temp;
}
/*
Both children exist.
Rotate the child having higher
priority upward.
*/

if (root->left->priority > root->right->priority) {
    root = rotateRight(root);

    root->right =
        deleteNode(root->right, task_id);
} else {
    root = rotateLeft(root);

    root->left =
        deleteNode(root->left, task_id);
}
}

return root;
}

```

```

/* Execute Highest Priority Task */
Node* executeTask(Node* root) {
    if (root == NULL) {
        printf("\nNo tasks available!\n");
        return NULL;
    }

    /*
        Since Treap follows MAX-HEAP
        property, root has the highest
        priority.
    */

    printf("\nExecuting Task:\n");
    printf("Task ID : %d\n", root->task_id);
    printf("Priority : %d\n", root->priority);

    /* Delete the root task */

    root = deleteNode(root, root->task_id);

    return root;
}

Node* search(Node* root, int task_id) {
    if (root == NULL)
        return NULL;

```

```
    if (root->task_id == task_id)
        return root;

    if (task_id < root->task_id)
        return search(root->left, task_id);

    return search(root->right, task_id);
}
```

```
void inorder(Node* root) {
    if (root == NULL)
        return;

    inorder(root->left);

    printf("Task ID: %d | Priority: %d\n",
        root->task_id,
        root->priority);

    inorder(root->right);
}
```

```
void preorder(Node* root) {
    if (root == NULL)
        return;

    printf("Task ID: %d | Priority: %d\n",
        root->task_id,
        root->priority);
```

```
    preorder(root->left);
    preorder(root->right);
}
```

```
/* free entire tree*/
void freeTree(Node* root) {
    if (root == NULL)
        return;

    freeTree(root->left);
    freeTree(root->right);

    free(root);
}
```

```
int main() {
    Node* root = NULL;

    int choice;
    int task_id;
    int priority;

    while (1) {
        printf("  PRIORITY TASK SCHEDULER\n");

        printf("1. Add Task\n");
        printf("2. Execute Highest Priority Task\n");
```

```
printf("3. Search Task\n");  
printf("4. Display Tasks (BST Order)\n");  
printf("5. Display Treap Structure\n");  
printf("6. Exit\n");
```

```
printf("\nEnter choice: ");  
scanf("%d", &choice);
```

```
switch (choice) {  
    case 1:  
        printf("Enter Task ID: ");  
        scanf("%d", &task_id);  
        printf("Enter Priority: ");  
        scanf("%d", &priority);  
        if (search(root, task_id) != NULL) {  
            printf("Task ID already exists!\n");  
        } else {  
            root = insert(root, task_id, priority);  
  
            printf("Task added successfully!\n");  
        }  
        break;  
  
    case 2:  
        root = executeTask(root);  
        break;  
  
    case 3:  
        printf("Enter Task ID to search: ");
```

```

scanf("%d", &task_id);
Node* found = search(root, task_id);
if (found != NULL) {
    printf("\nTask Found!\n");
    printf("Task ID : %d\n", found->task_id);
    printf("Priority : %d\n", found->priority);
} else {
    printf("\nTask not found!\n");
}
break;

```

case 4:

```

if (root == NULL) {
    printf("\nNo tasks available!\n");
} else {
    printf("\nTasks in BST order:\n");

    inorder(root);
}
break;

```

case 5:

```

if (root == NULL) {
    printf("\nNo tasks available!\n");
} else {
    printf("\nTreap Preorder:\n");

    preorder(root);
}

```

```
break;
```

```
case 6:
```

```
    freeTree(root);
```

```
    printf("\nProgram terminated.\n");
```

```
    return 0;
```

```
default:
```

```
    printf("\nInvalid choice!\n");
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```